Google Kubernetes Engine (GKE)

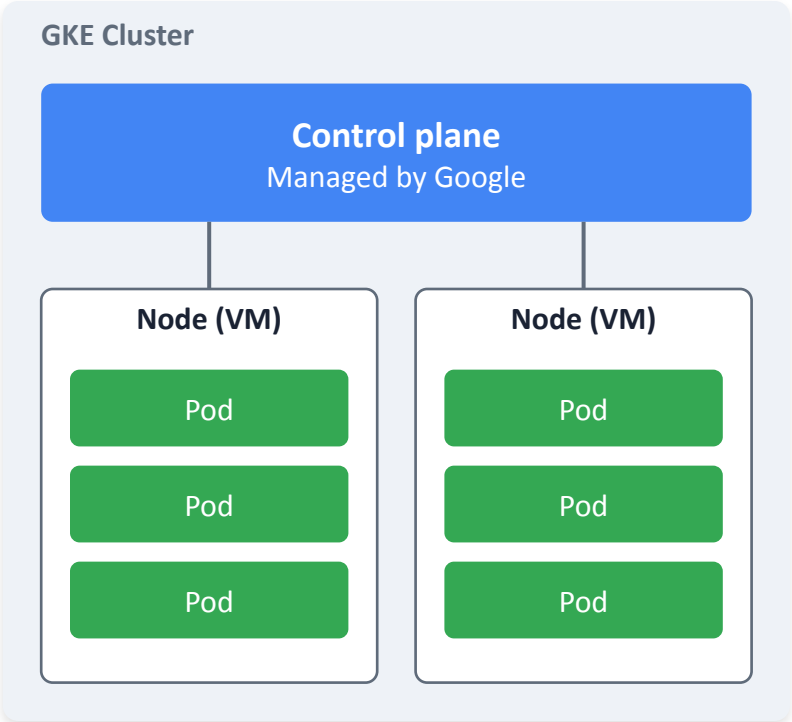
Clusters, Deployments, Services and Persistent Storage



What is GKE?

GKE is Google Cloud's managed Kubernetes service for deploying, managing and scaling containerized applications.

- Google runs the control plane: API server, etcd and scheduler
- Your workloads run on worker nodes (Compute Engine VMs)
- Built on open-source Kubernetes, which Google created
- Integrated with IAM, VPC, Cloud Logging and Monitoring



Why do we need GKE?



Self-managed Kubernetes

- Install and secure the control plane yourself
- Back up and maintain etcd
- Patch and upgrade every node by hand
- Build high availability across zones
- Wire up logging, monitoring and scaling



With GKE

- Google-managed control plane with an SLA
- Auto-upgrades and node auto-repair
- Regional clusters for built-in HA
- Automatic Pod and node scaling
- Native Logging, Monitoring and IAM

Advantages of GKE

Fully managed

Google runs, patches and upgrades the control plane

Autoscaling

Scale pods and nodes automatically with demand

Built-in security

Workload Identity, Shielded nodes, private clusters

High availability

Regional clusters spread across multiple zones

Observability

Cloud Logging and Monitoring out of the box

Cost efficiency

Autopilot pay-per-pod billing and Spot VMs

Types of GKE Clusters



MODE OF OPERATION

Autopilot

Google manages nodes, scaling and security. Pay per pod resource request. Recommended default.



MODE OF OPERATION

Standard

You manage node pools, machine types and upgrades. Pay per node VM. Full control.

AVAILABILITY

Zonal

Control plane in one zone; nodes in one or more zones

AVAILABILITY

Regional

Control plane and nodes replicated across zones

NETWORK

Private

Nodes get internal IPs only; restricted API access

What is a Deployment?

A Deployment declares the desired state for a set of identical Pods, and Kubernetes keeps the cluster matching it.



Replicas
Keeps N Pods running via a ReplicaSet



Rolling updates
Upgrade images with zero downtime



Rollbacks
Revert to any previous revision



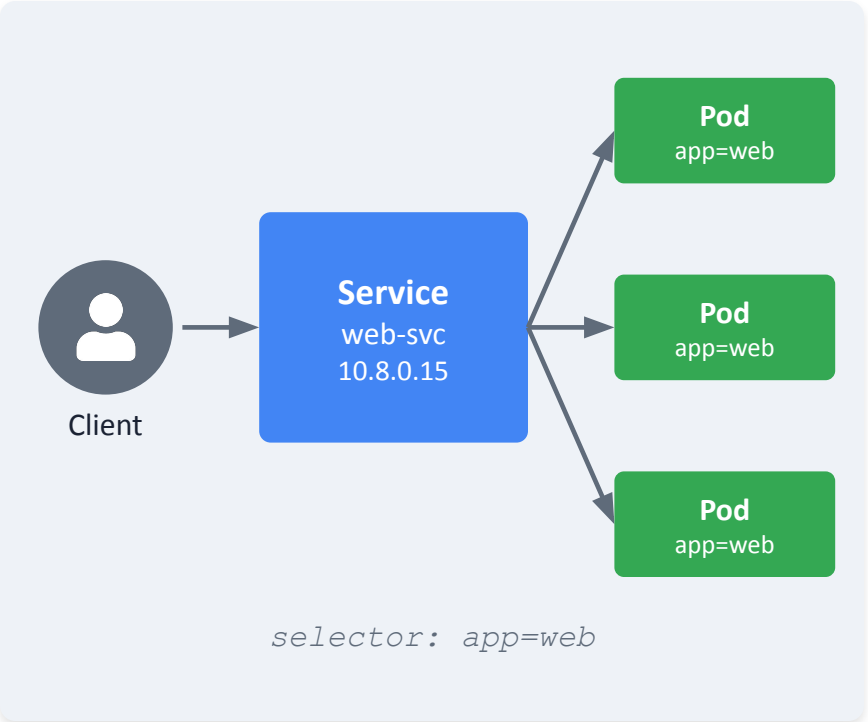
Self-healing
Replaces failed or deleted Pods

```yaml
# deployment.yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: web-app
spec:
  replicas: 3
  selector:
    matchLabels:
      app: web
  template:
    metadata:
      labels:
        app: web
    spec:
      containers:
      - name: web
        image: nginx:1.27
        ports:
        - containerPort: 80
```

What is a Service?

Pods are ephemeral and their IPs change. A Service gives them one stable network endpoint.

- Stable virtual IP and DNS name
- Finds Pods using label selectors
- Load-balances traffic across healthy Pods
- Decouples clients from the Pod lifecycle



Types of Kubernetes Services



ClusterIP

Default · internal

Internal IP for traffic inside the cluster only



NodePort

Node IP : port

Opens a static port (30000–32767) on every node



LoadBalancer

Public external IP

Provisions a Google Cloud load balancer



ExternalName

DNS alias

Maps the Service to an external DNS name

Also know: Headless Service (clusterIP: None) for StatefulSets, and Ingress for HTTP(S) routing

Lab: Deploy & Expose an App

1. **Create an Autopilot cluster**

```
gcloud container clusters create-auto demo-cluster --region=asia-south1
```

2. **Connect kubectl to the cluster**

```
gcloud container clusters get-credentials demo-cluster --region=asia-south1
```

3. **Create a Deployment with 3 replicas**

```
kubectl create deployment web-app --image=nginx --replicas=3
```

4. **Expose it with a LoadBalancer Service**

```
kubectl expose deployment web-app --type=LoadBalancer --port=80
```

5. **Verify, then open http://EXTERNAL-IP in a browser**

```
kubectl get pods,svc -w
```

6. **Clean up: Service first, then the cluster**

```
kubectl delete svc web-app; gcloud container clusters delete demo-cluster --region=asia-south1
```

What are PV and PVC?

SUPPLY: the actual storage

PersistentVolume (PV)

- Cluster-wide storage resource
- Created by an admin or dynamically
- Backed by a Persistent Disk on GKE
- Outlives any single Pod

binds

DEMAND: a request for storage

PersistentVolumeClaim (PVC)

- Created by the application team
- Asks for size and access mode
- Binds to a matching PV
- Mounted by Pods as a volume



Dynamic Volume Provisioning

STEP 1
PVC is created

STEP 2
StorageClass picks provisioner

STEP 3
PD CSI driver creates a disk

STEP 4
PV is created and bound

STEP 5
Pod mounts the volume

```
# pvc.yaml
apiVersion: v1
kind: PersistentVolumeClaim
metadata:
  name: data-pvc
spec:
  storageClassName: standard-rwo
  accessModes: ["ReadWriteOnce"]
  resources:
    requests:
      storage: 10Gi
```



Static
Admin creates PVs up front; claims bind to them



Dynamic
GKE creates a disk for every new PVC



GKE default
standard-rwo class (Balanced Persistent Disk)

Persistent Storage Use Cases



Databases
MySQL, PostgreSQL, MongoDB on StatefulSets



CMS and file uploads
WordPress media and user-uploaded files



Messaging and streaming
Kafka and RabbitMQ brokers keep their data



ML and data workloads
Training data, model checkpoints, notebooks



CI/CD and caching
Jenkins home, build caches, Redis snapshots



On-demand environments
Each new PVC gets its own fresh disk

Key Takeaways



GKE is managed Kubernetes on Google Cloud, in Autopilot or Standard mode



Deployments run and heal Pods; Services give them a stable endpoint



PVCs request storage; StorageClasses provision disks dynamically

Thank you · Questions?